Fachhochschule Nordwestschweiz
Hochschule für Technik

Imagenet Architectures

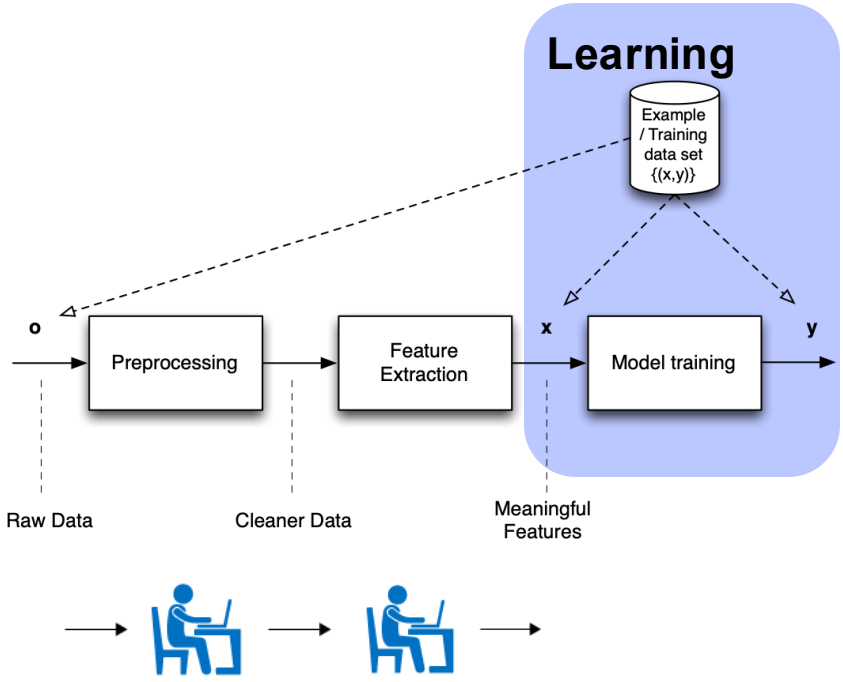
Deep Learning

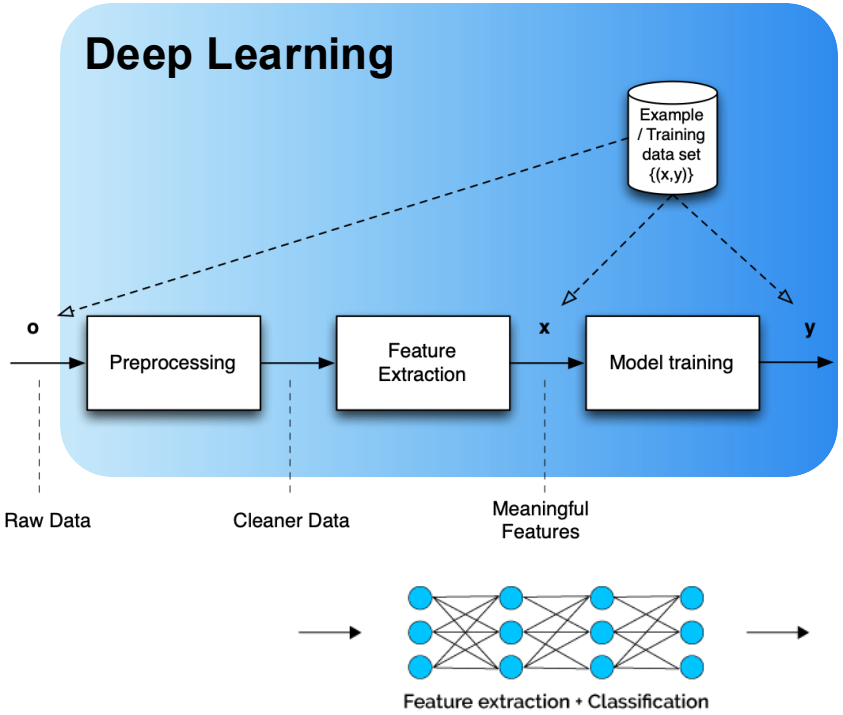
HS 2025

Martin Melchior

Convolution layers are extracting features

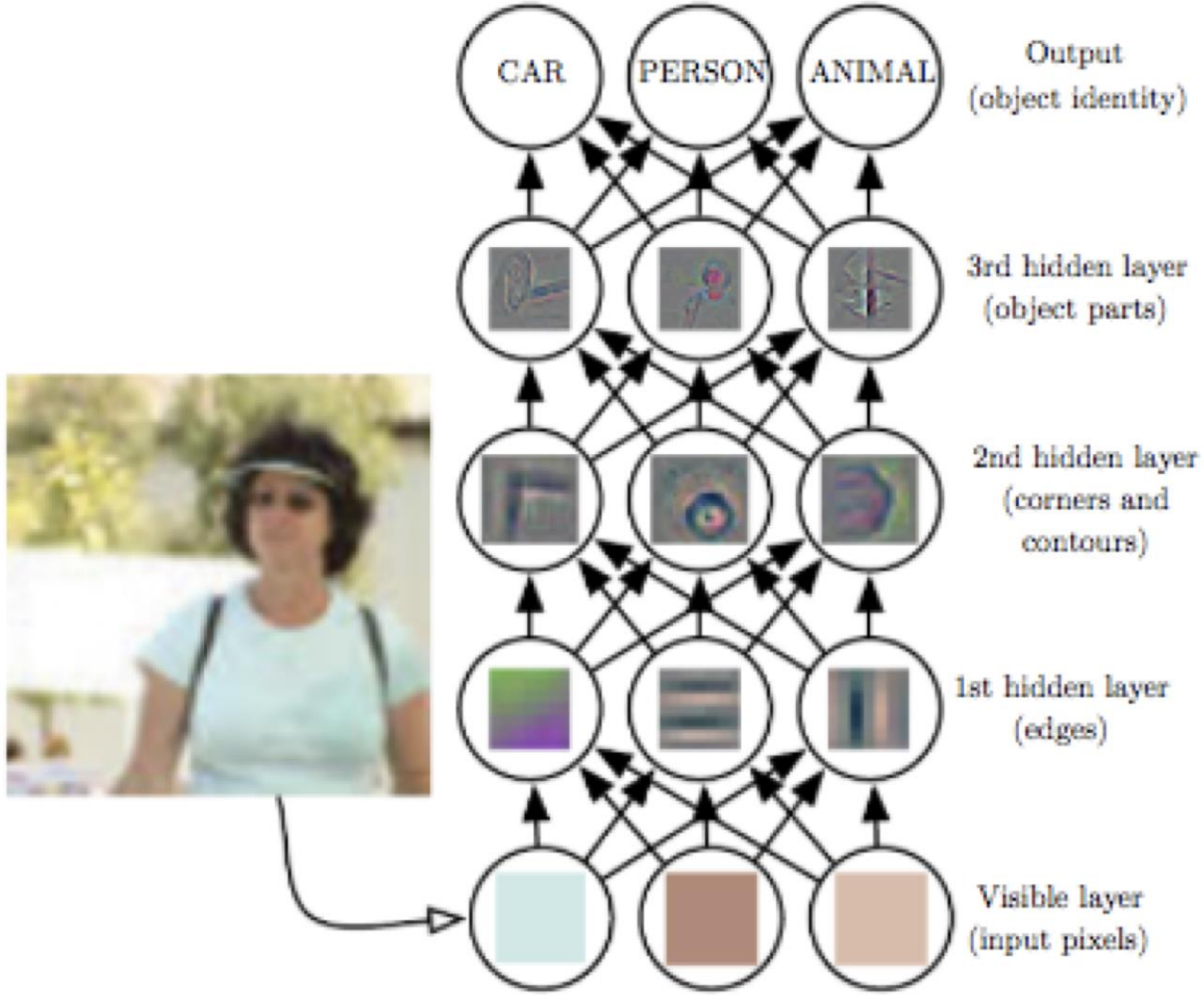
- The filter parameters are learnt through the training process. The whole system is learning not only to classify but also to extract features.



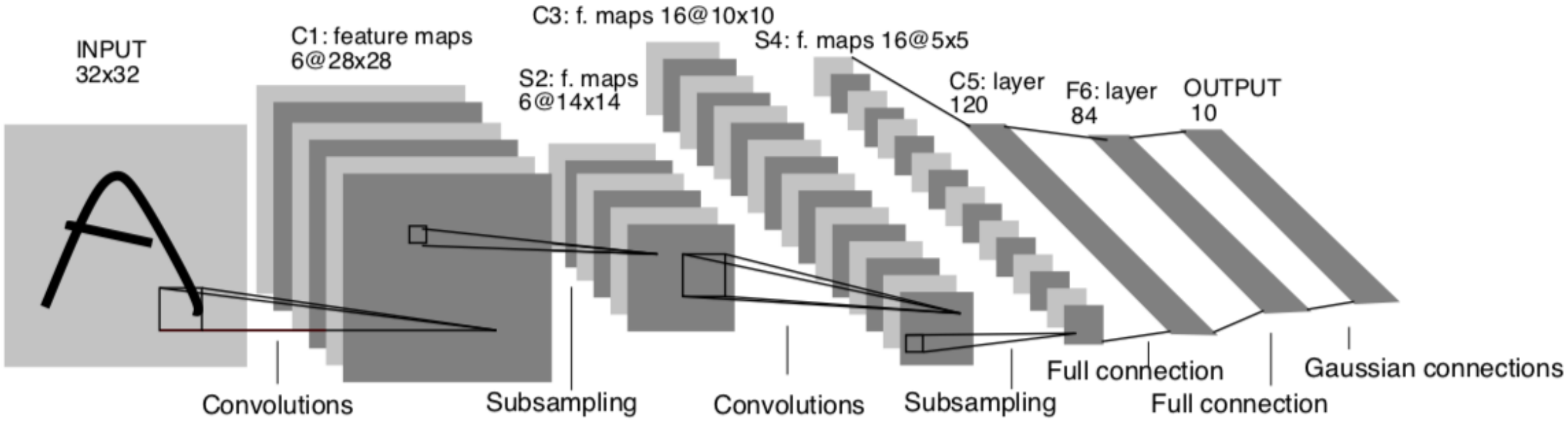


Stacked CNN Layers

- Stacked CNN Layers learn a hierarchy of features.
- Can be validated by checking the learned filters.
- Increase depth!



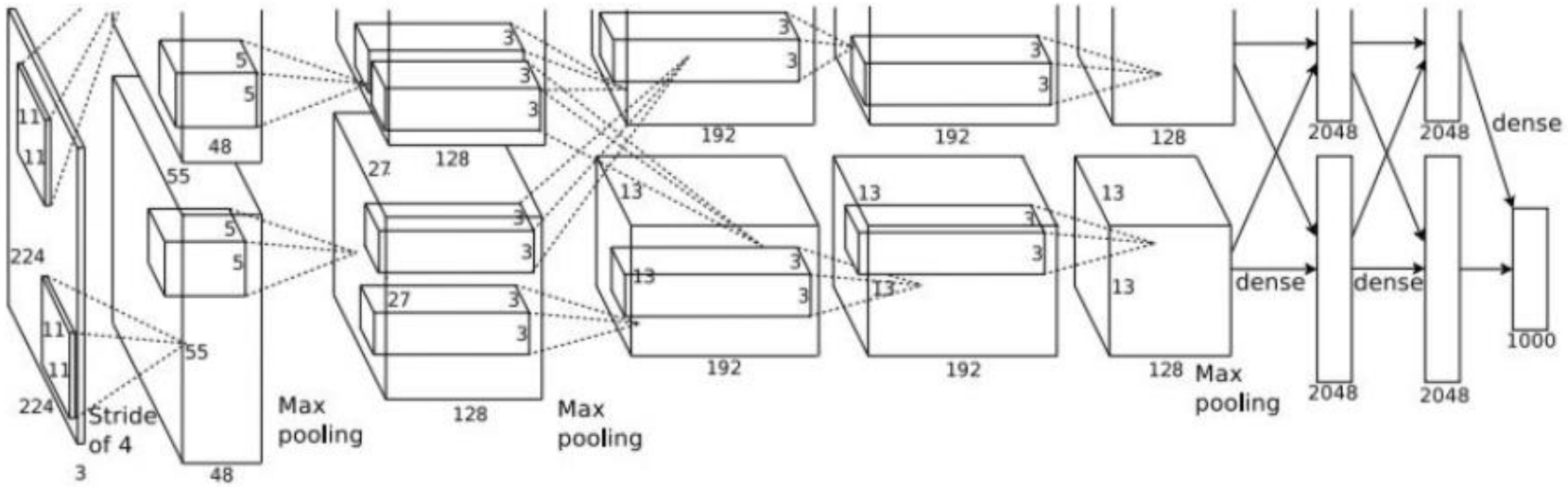
LeNet-5 (1998)



Conv filters were 5x5, applied at stride 1
Subsampling (Pooling) layers were 2x2 applied at stride 2
i.e. architecture is [CONV-POOL-CONV-POOL-FC-FC]

- LeNet. The first successful applications of Convolutional Networks were developed by **Yann LeCun** in 1990's. Of these, the best known is the LeNet architecture that was used to read zip codes, digits, etc.
- http://yann.lecun.com/exdb/publis/pdf/lecun-98.pdf

AlexNet (2012)

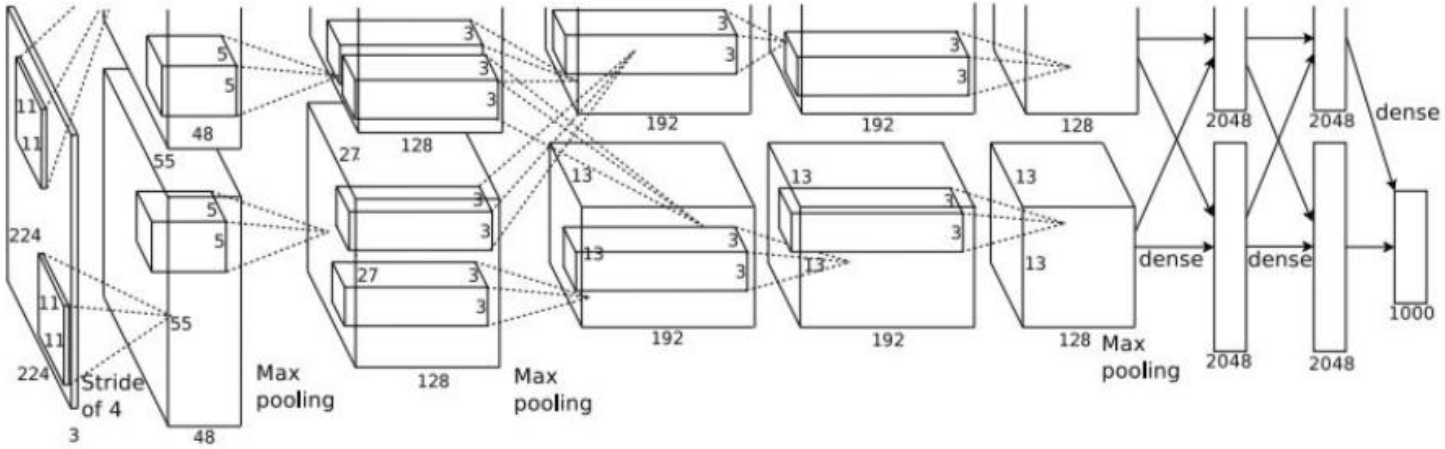


Two parallel streams for optimizing memory

Architecture:
CONV1
MAX POOL1
NORM1
CONV2
MAX POOL2
NORM2
CONV3
CONV4
CONV5
Max POOL3
FC6
FC7
FC8

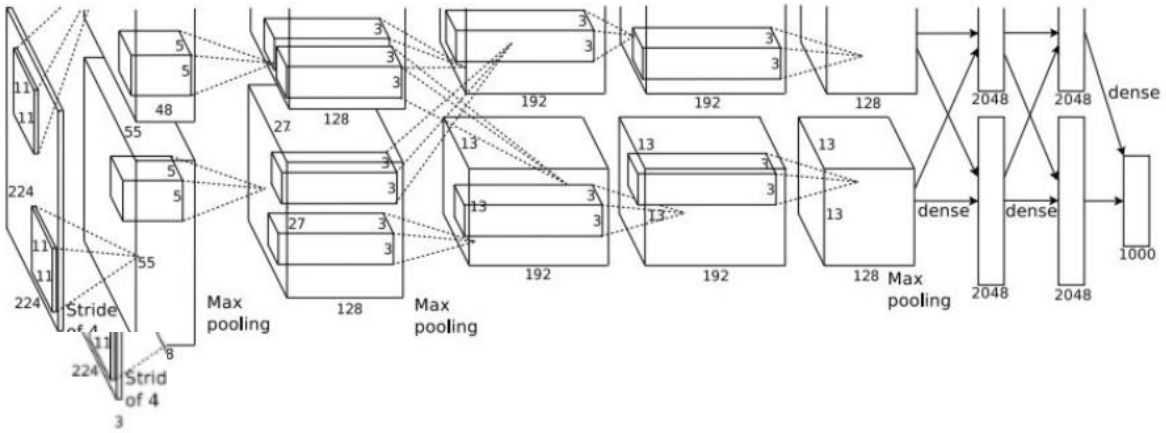
- The first work that popularized Convolutional Networks in Computer Vision was the AlexNet, developed by **Alex Krizhevsky**, I. Sutskever and G. Hinton.
- The AlexNet was submitted to the ImageNet ILSVRC challenge in 2012 and significantly outperformed the second runner-up:
  top 5 error of 16% compared to runner-up with 26% error
- The Network had a very similar architecture to LeNet, but was deeper, bigger, and featured Convolutional Layers stacked on top of each other (previously it was common to only have a single CONV layer always immediately followed by a POOL layer).
- http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks

AlexNet



- Input : 227x227x3 images
- First layer (CONV1): 96 filters 11x11, S = 4
  - Activation map size = (227-11)/4 + 1 = 55x55
  - Output volume = (55x55x96):
  - Number of parameters: 96x(11x11x3) + 96 = 34’944 ~ 35K
- Second layer (POOL1): 3x3, S = 2
  - activation map size = (55-3)/2 + 1 = 27x27
  - output volume = 27x27x96
- …

AlexNet

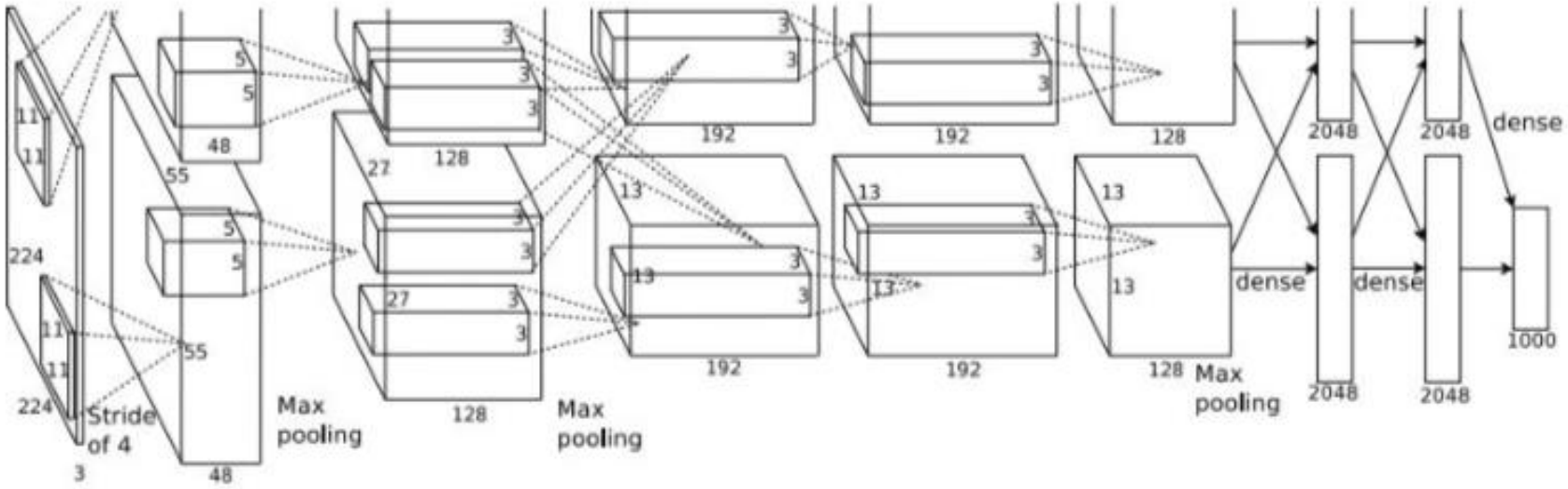


Full (simplified) AlexNet architecture:
[227x227x3] INPUT
[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer
[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer
[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2
[4096] FC6: 4096 neurons
[4096] FC7: 4096 neurons
[1000] FC8: 1000 neurons (class scores)

Some Details:

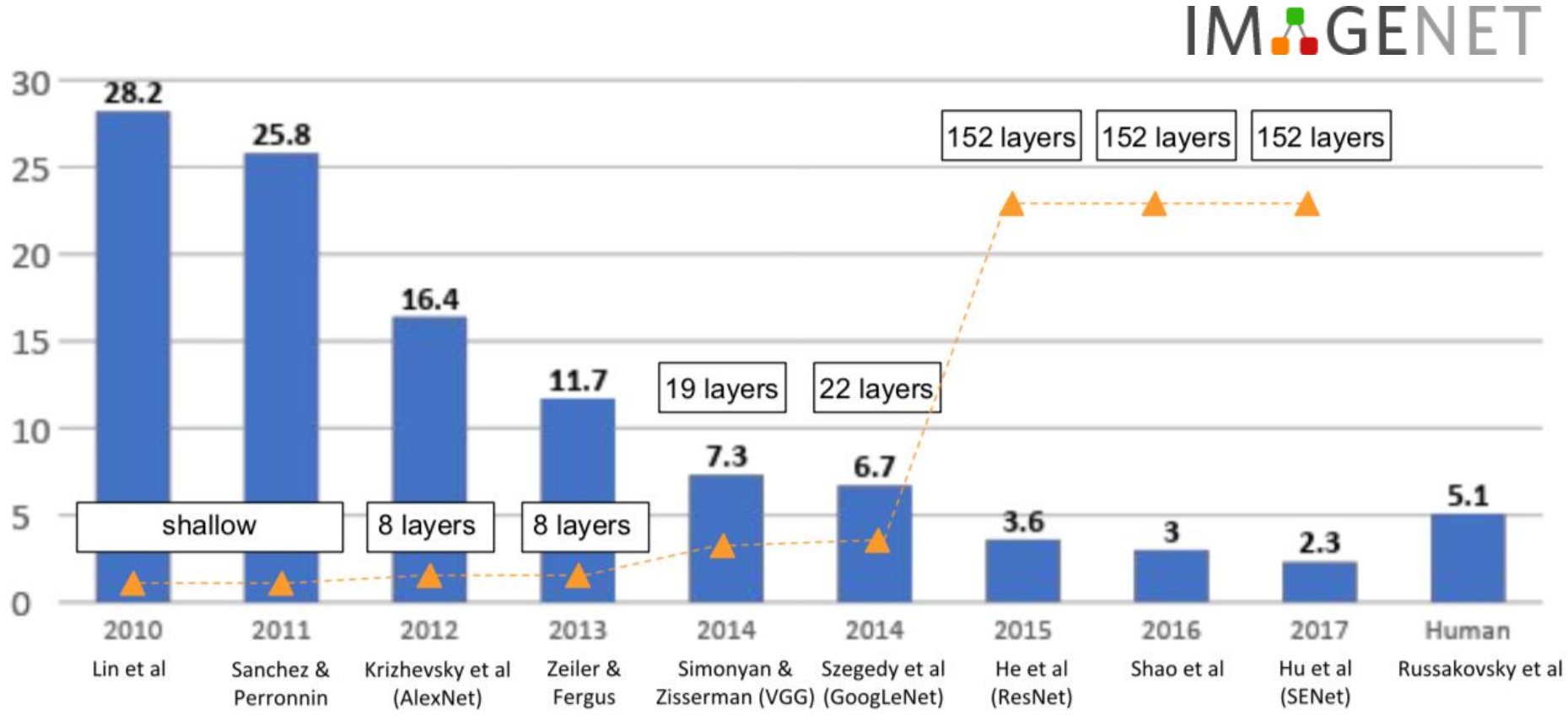
- ReLU (first use)
- Used Norm layers (not same as BatchNorm)
- heavy data augmentation
- dropout 0.5
  batch size 128
  SGD Momentum 0.9
  Learning rate 1e-2, reduced by 10 manually when val accuracy plateaus
  L2 weight decay 5e-4

Hierarchy of Features

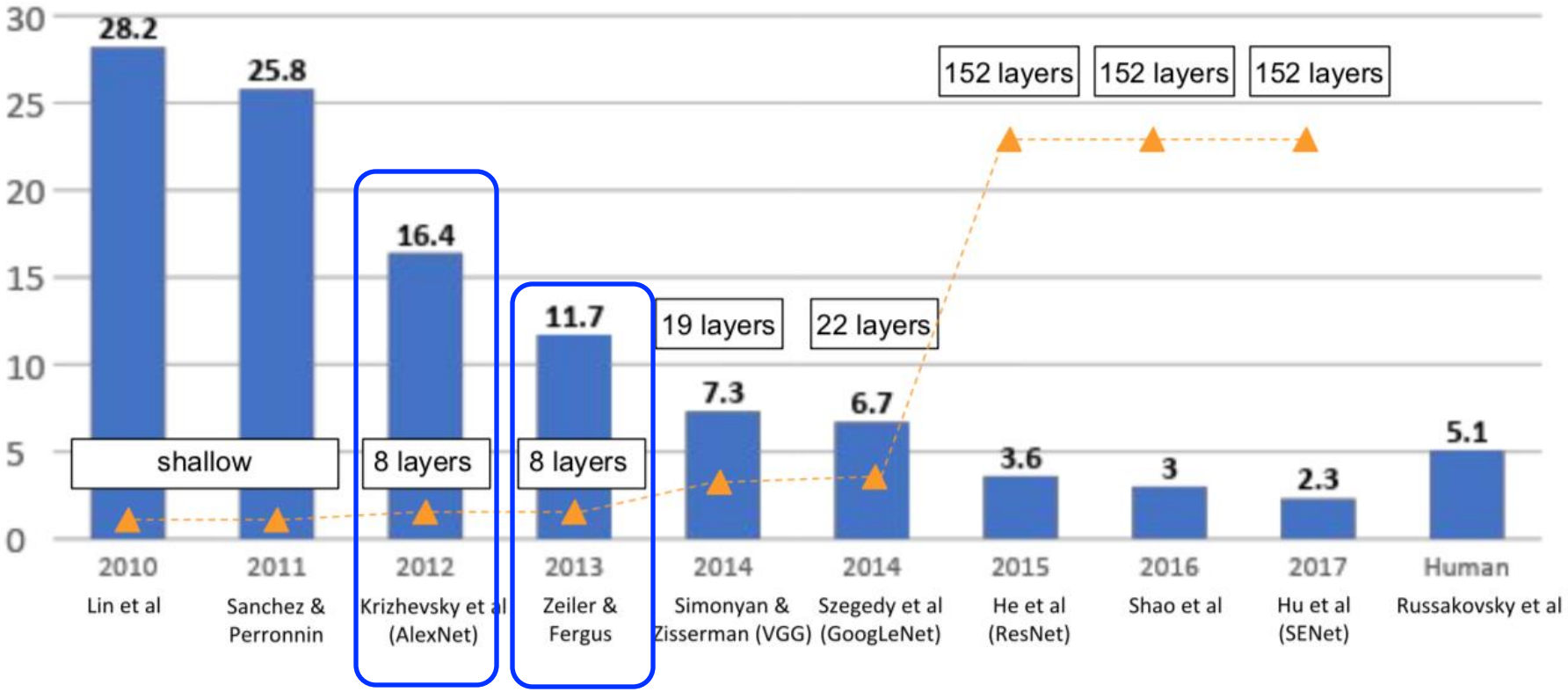


- The deeper the network:
  - The bigger are the number of filters in the conv layers
  - The smaller are the activation maps due to max pooling
- Intuitively:
  - Early layers will extract lower-level features
  - Late layers will extract higher-level features

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) - winners

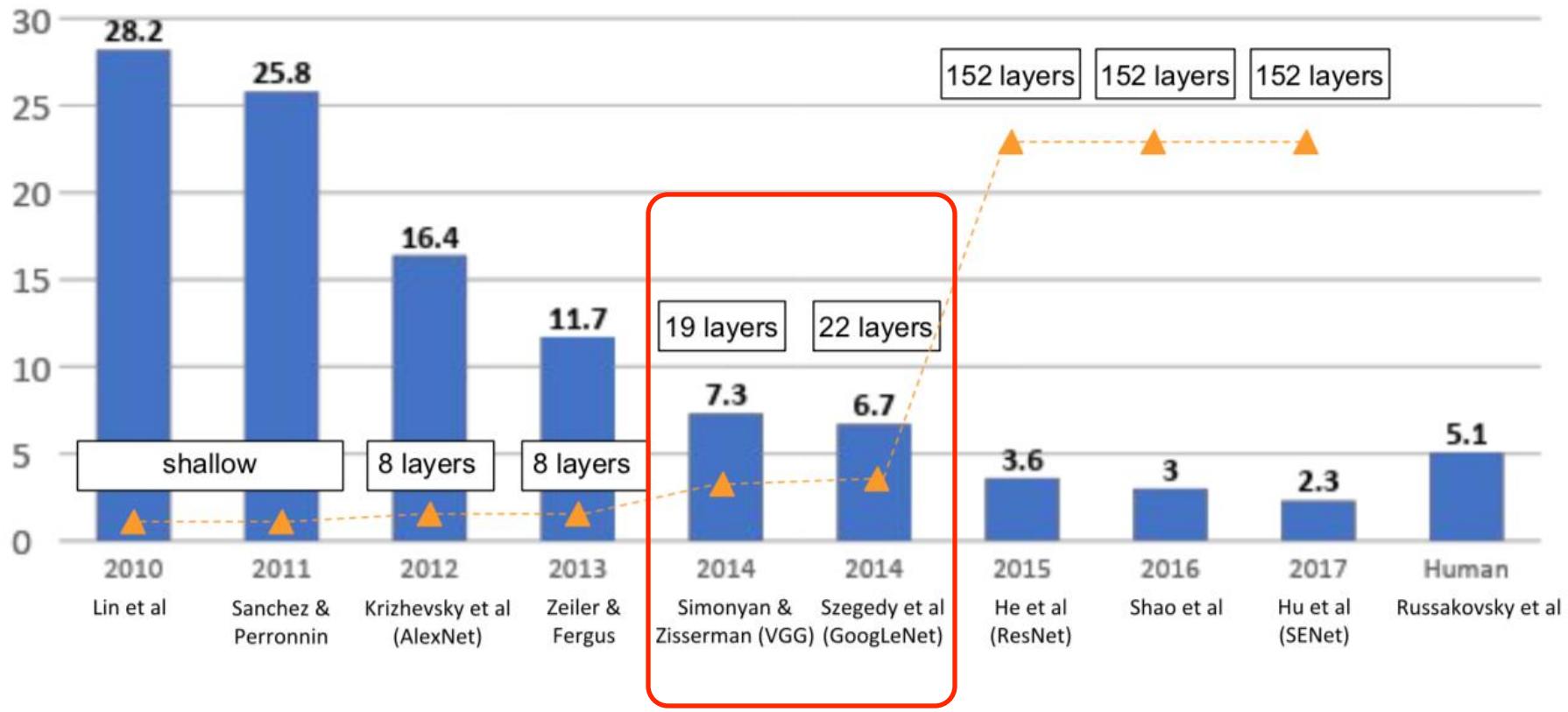


https://www.image-net.org/challenges/LSVRC/



AlexNet : First deep learning based approach winner of the competition.

ZFNet : similar to AlexNet but with hyper-parameter tuning.



Deeper networks

VGGNet (2014)

Simonyan and Zisserman, 2014

Stack of three 3x3 conv (stride 1) layers has same effective receptive field as one 7x7 conv layer.

But deeper, more non-linearities.

Small filters, Deeper networks

From 8 layers (AlexNet) to 16 - 19 layers (VGGNet)

Only 3x3 CONV S=1, P=1 and 2x2 MAX POOL S=2

7.3% top 5 error in ILSVRC'14

VGGNet

Most memory usage in CONV layers

(not counting biases)

INPUT: [224x224x3] memory: 224*224*3=150K params: 0
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*64)*64 = 36,864
POOL2: [112x112x64] memory: 112*112*64=800K params: 0
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456
POOL2: [56x56x128] memory: 56*56*128=400K params: 0
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
POOL2: [28x28x256] memory: 28*28*256=200K params: 0
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512] memory: 14*14*512=100K params: 0
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512] memory: 7*7*512=25K params: 0
FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = 102,760,448
FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

Most parameters in FC layers

TOTAL memory: 24M * 4 bytes ~= 96MB / image (only forward! ~*2 for bwd)
TOTAL params: 138M parameters

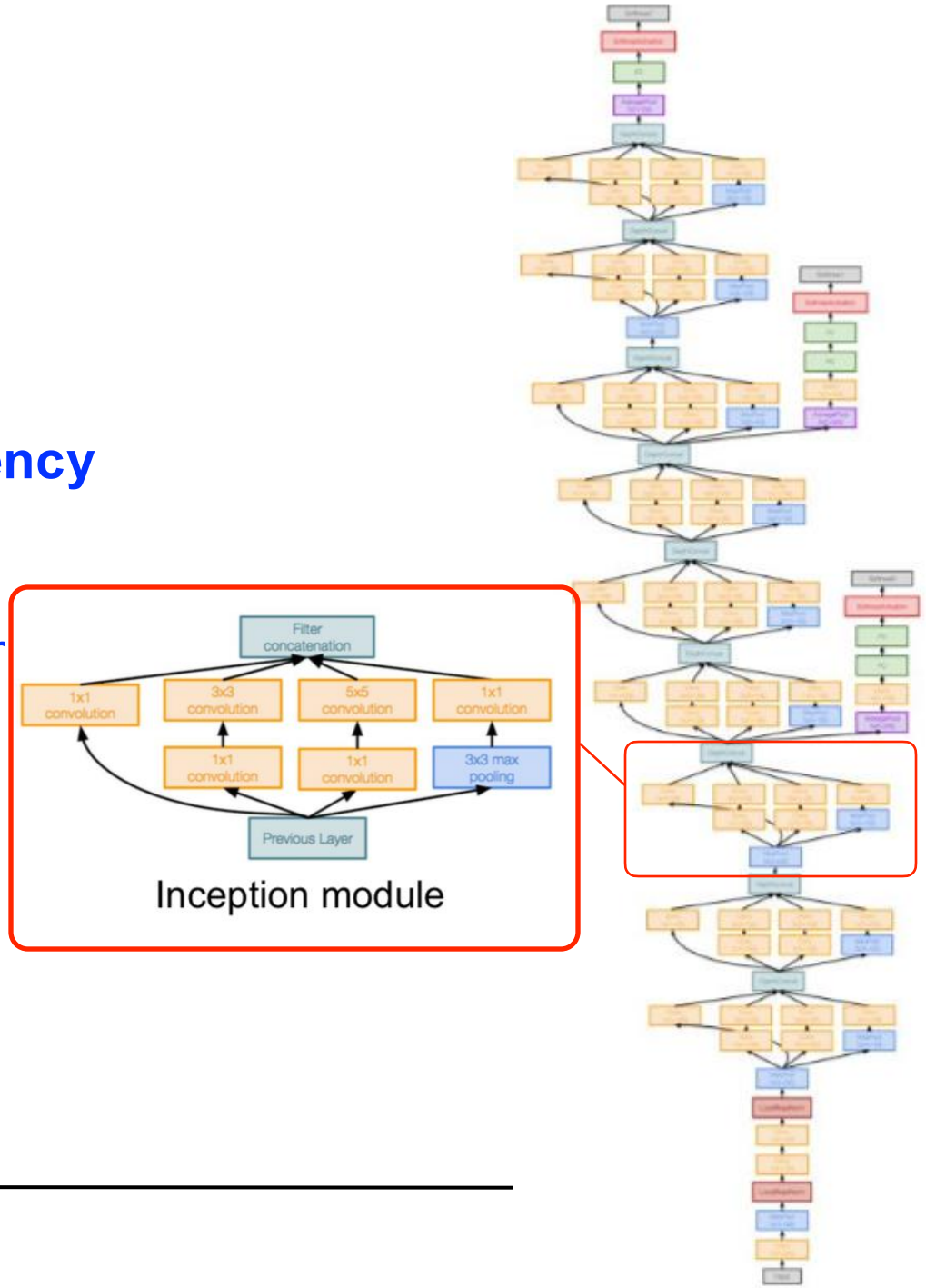
VGG16

Common names

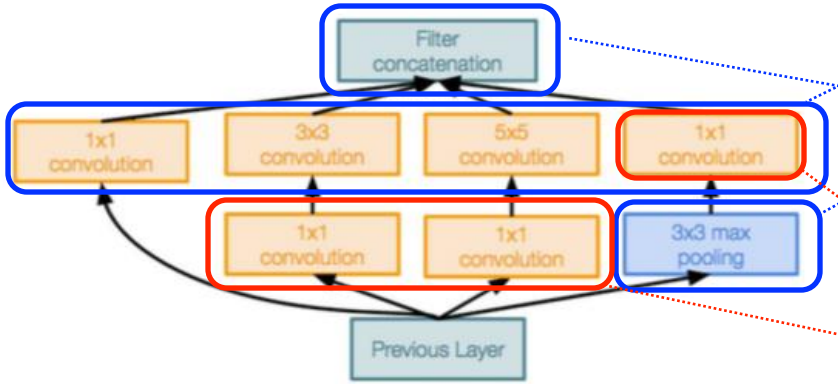
GoogLeNet (2014)

Szegedy et al., 2014

- **Deeper networks, with computational efficiency**
  - 22 layers
  - ”Network in the network” efficient “**Inception**” r
  - No FC layers
  - Only 5 million parameters!
    12x less than AlexNet
  - ILSVRC’14 classification winner
    (6.7% top 5 error)



GoogLeNet



Inception module

Intuition: mix filter size to allow looking for more or less details from the previous layer, i.e. related to the receptive field size. We don't know in advance what is the best size, e.g. close or far object.

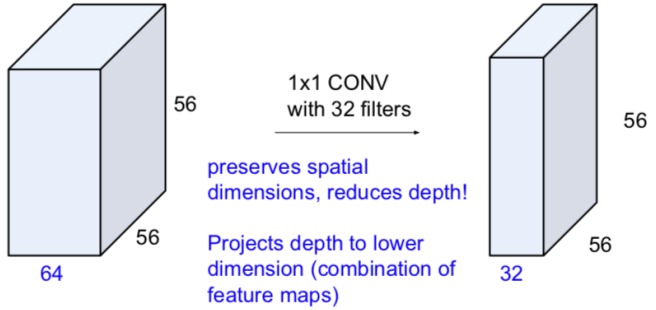
So let's use **in parallel** different filter size, (1x1, 3x3, 5x5).

Ultimately no filter but pooling operation (3x3).

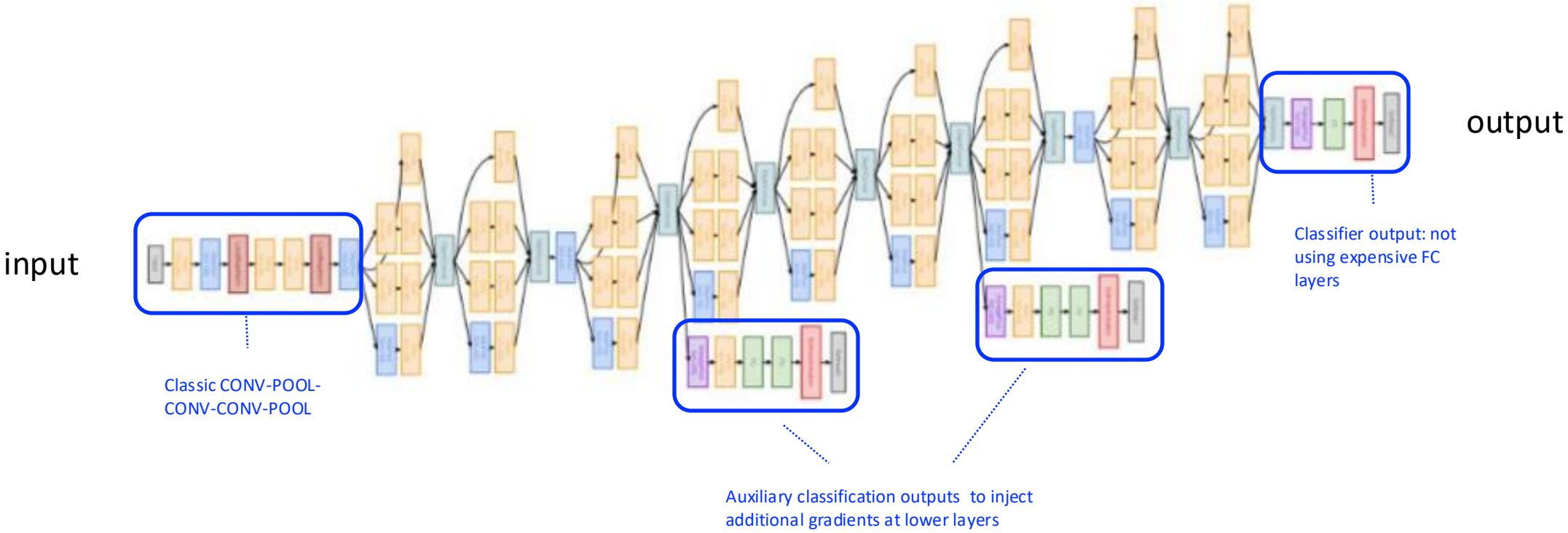
Then concatenate all filter outputs together depth-wise.

To reduce the number of operation, 1x1 conv are used. So-called **bottleneck layers**. (see slide 37 of CNN-part 1 week class)

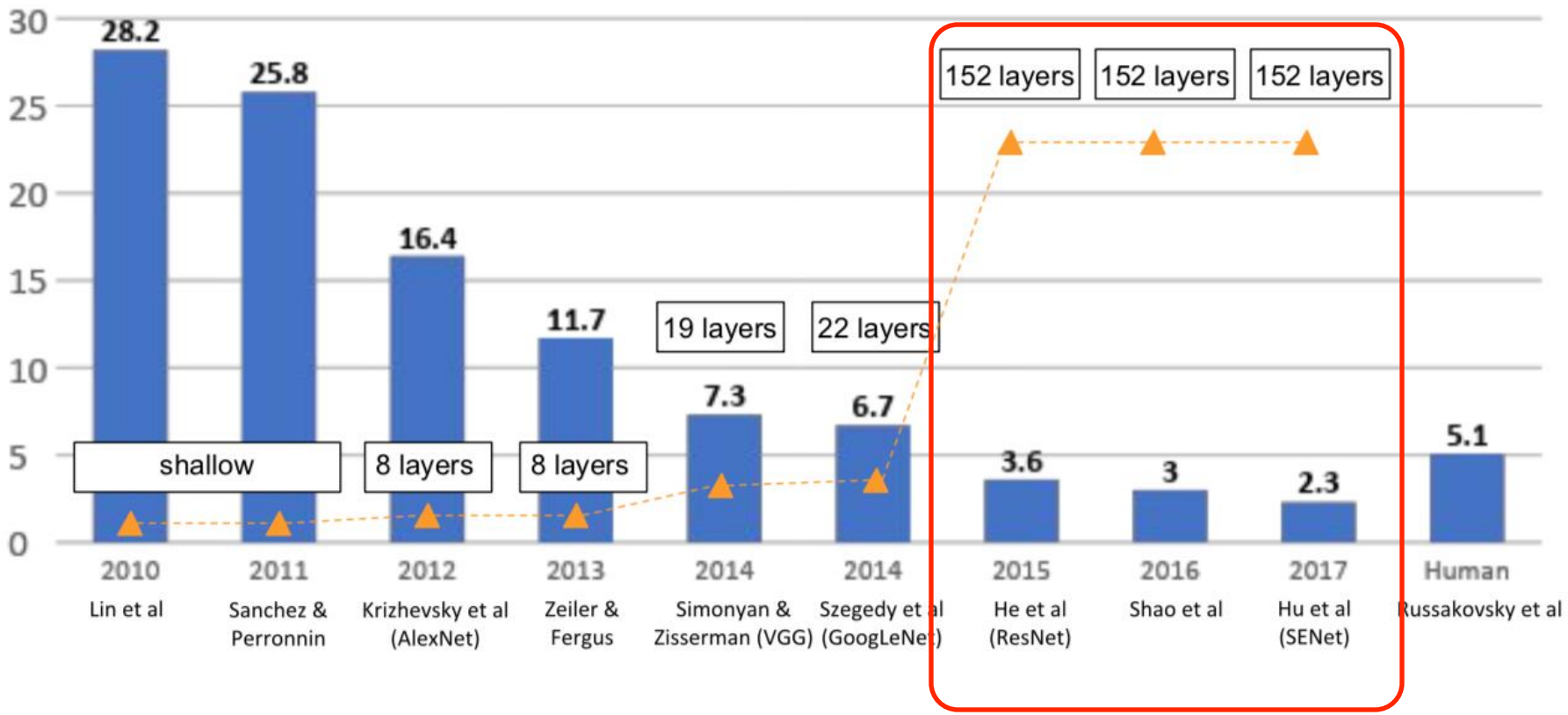
Reminder: 1x1 convolutions



GoogLeNet



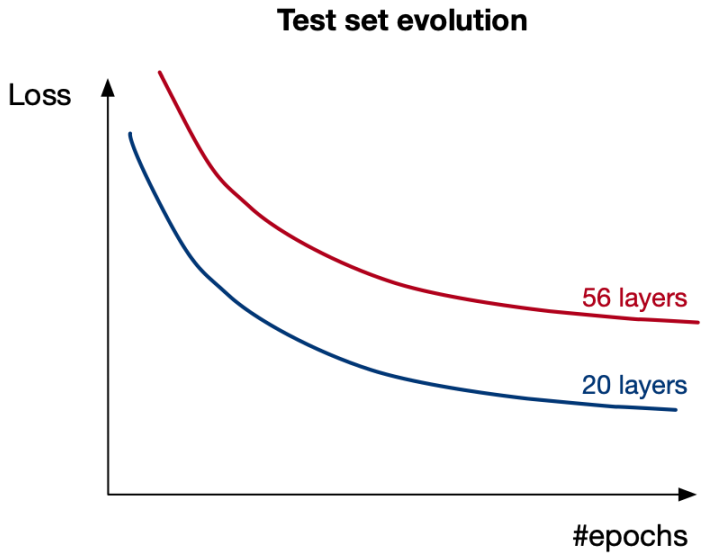
The architecture becomes non-sequential and complex !

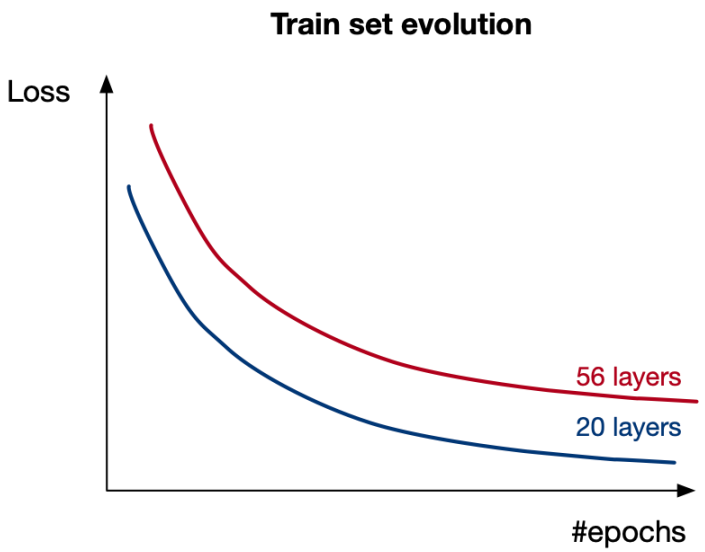


Revolution of depth !

ResNet (2015)

- He et al., 2015
- Observation they did: from AlexNet and VGGNet, increasing the number of layers did not succeed. They observed something like this:



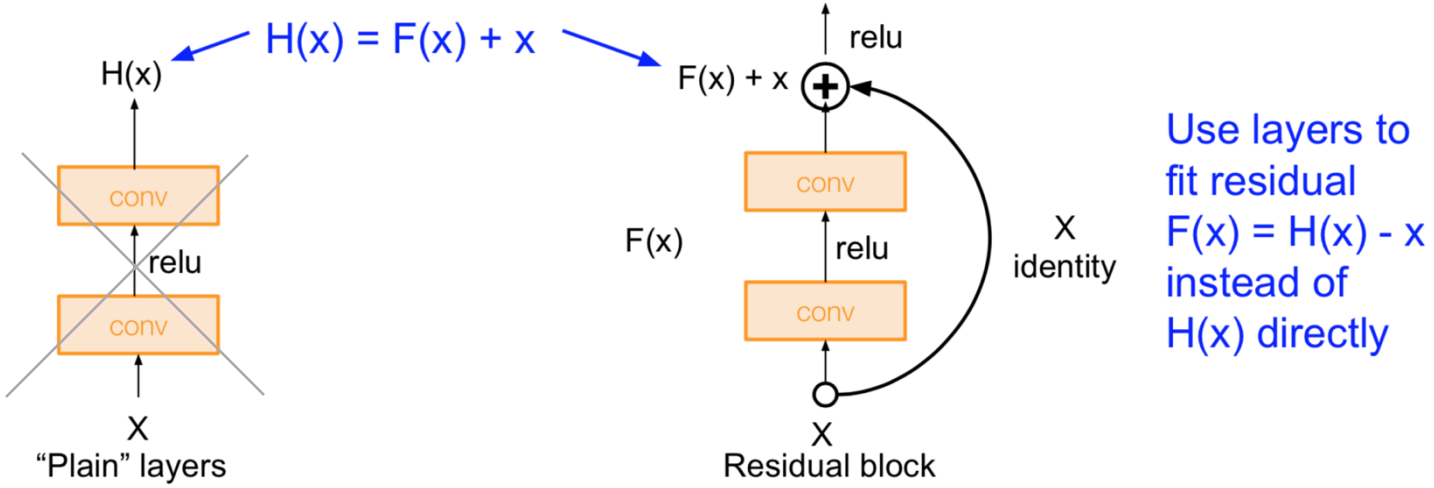


56-layer model performs worse on both training and test error.

Deeper model performs worse, but it's not caused by overfitting!

ResNet

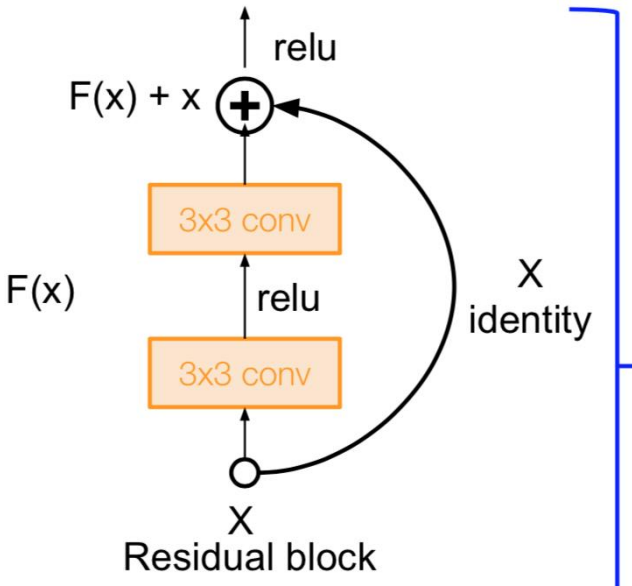
- Hypothesis: the problem is an optimization problem, deeper models are more difficult to optimize
- Ideas:
  - Deeper layers should be able to “fall back” to identity mapping if difficulties to converge
  - Easier to model a “delta” from one layer to the other than a full feature
- Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

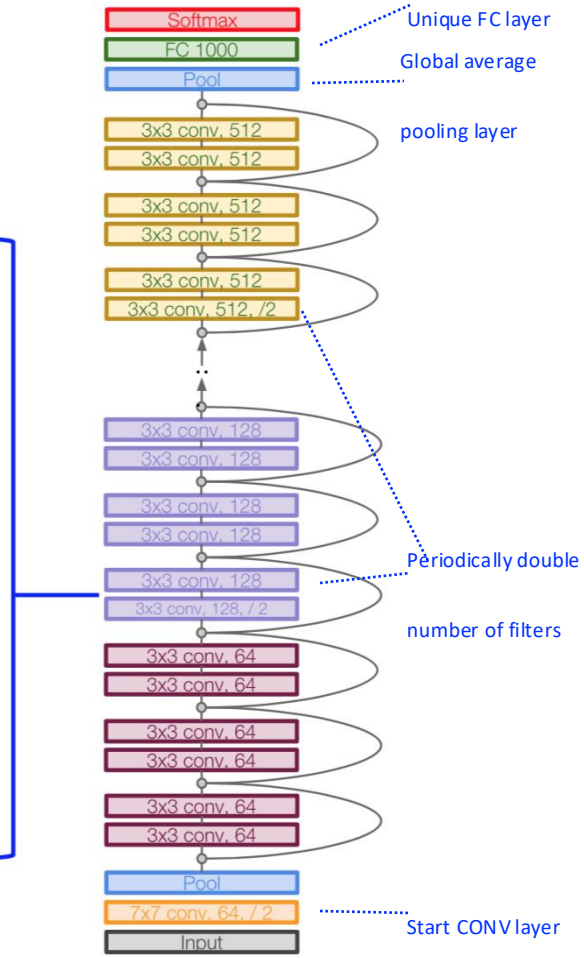


ResNet

Very deep networks using residual connections

- 152-layer model for ImageNet
- ILSVRC'15 classification winner (3.57% top 5 error)
- Swept all classification and detection competitions in ILSVRC'15 and COCO'15!

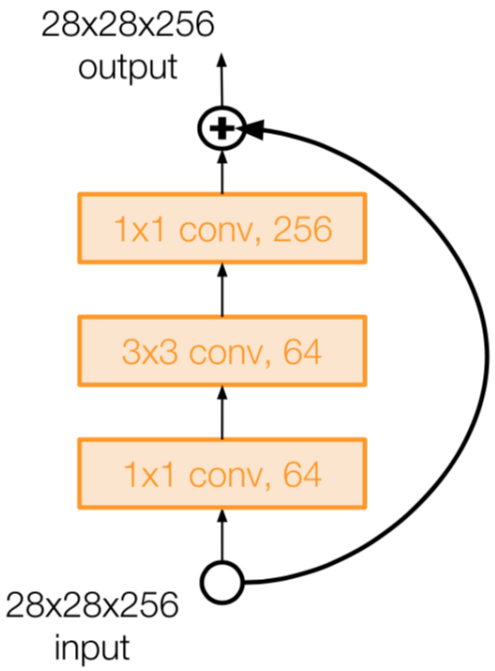


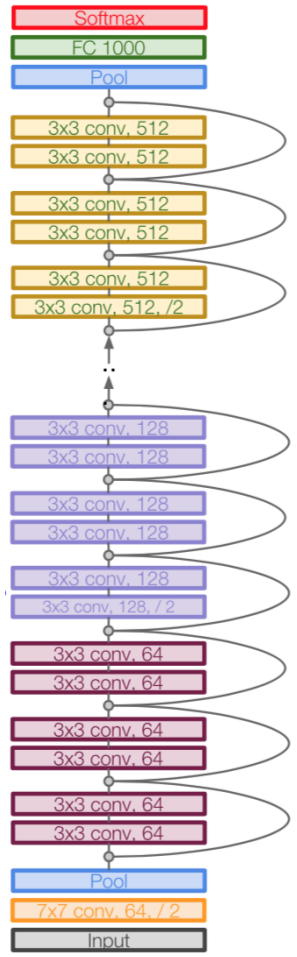


ResNet

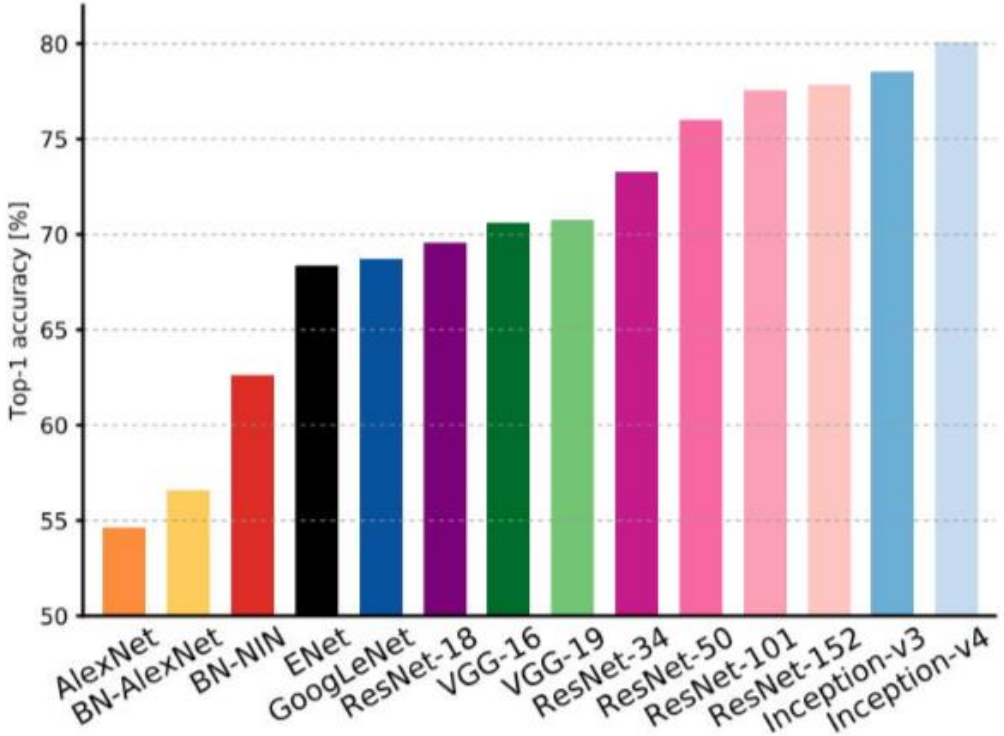
Total depth:
152 layers for imageNet competition !

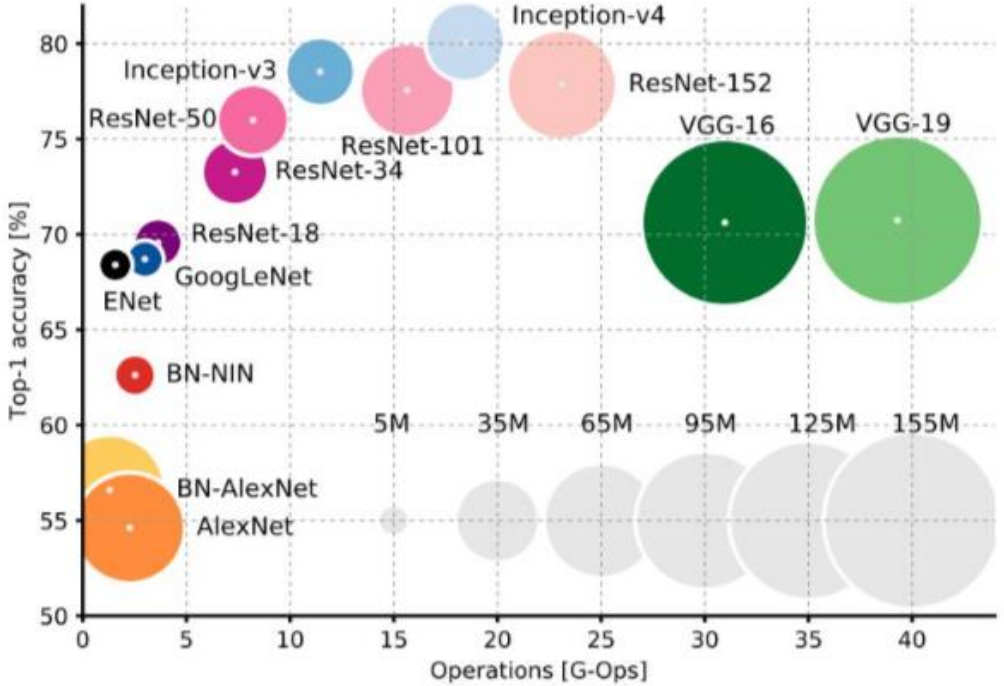
- Trick to go deeper than 50 layers:
  - Use bottleneck layers 1x1x64 to reduce the number of operations and parameters
- Other training strategies:
  - Batch Normalization after every CONV layer
  - Xavier 2/ initialization from He et al.
  - SGD + Momentum (0.9)
  - Learning rate: 0.1, divided by 10 when validation error plateaus
  - Mini-batch size 256
  - Weight decay of 1e-5
  - No dropout used





Comparing complexity





An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Progress in ImageNet Classification after 2017

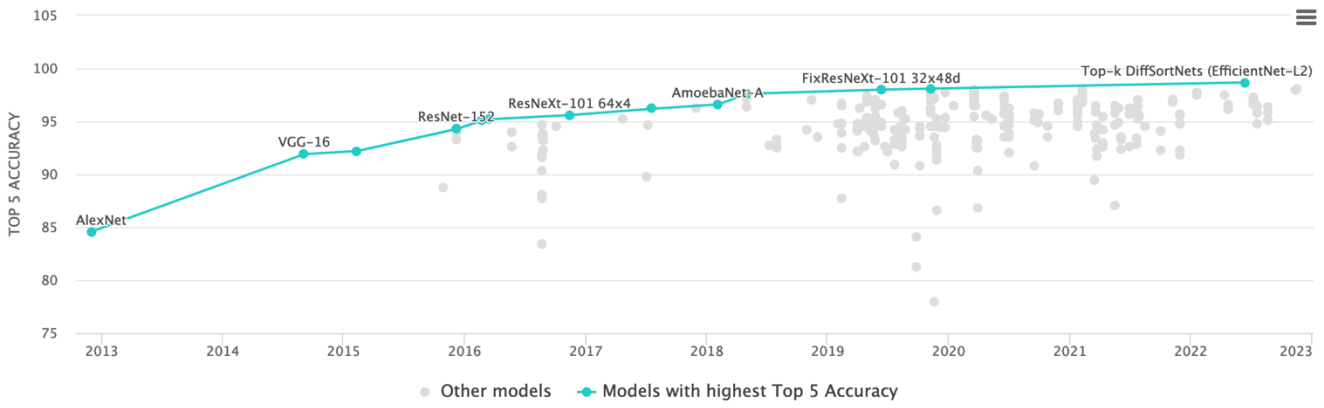
Image Classification

Image Classification on ImageNet

[paperswithcode]

Leaderboard Dataset

View Top 5 Accuracy by Date for Models not using extra training data



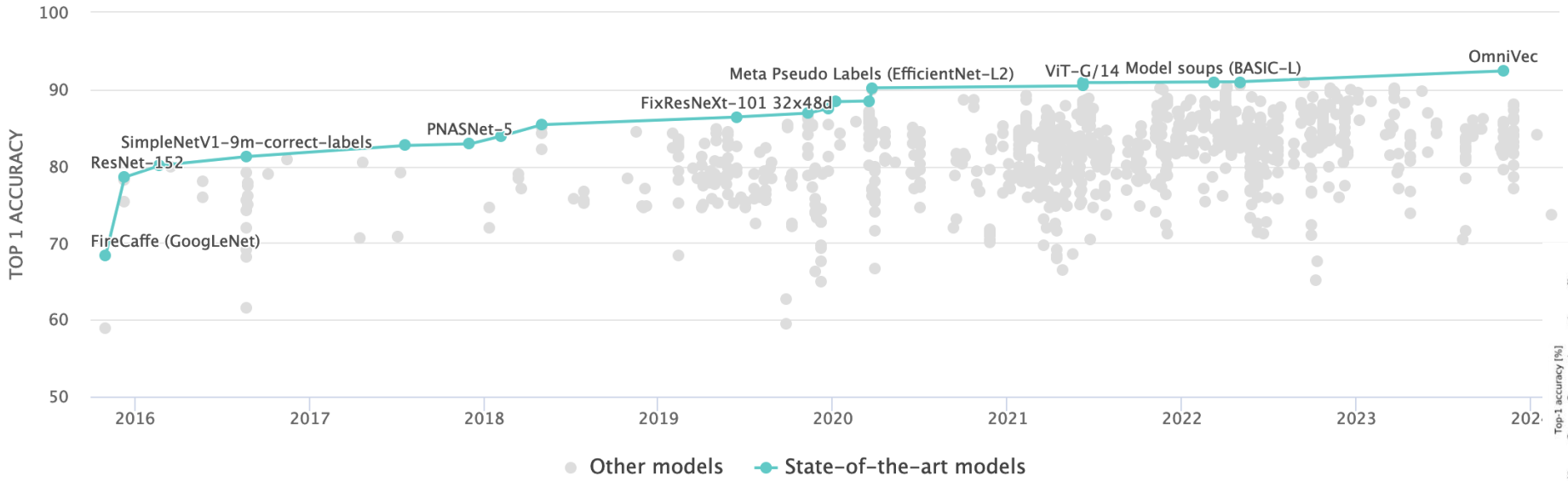
- Based on EfficientNet: 2.9%, with pre-training strategies 1.4%
- Based on ResNeXt: 4.4% (in 2016), with retraining strategies 2.0%
- Based on ViT: 2.5%, with retraining 1.3%

Image Classification on ImageNet

Leaderboard Dataset

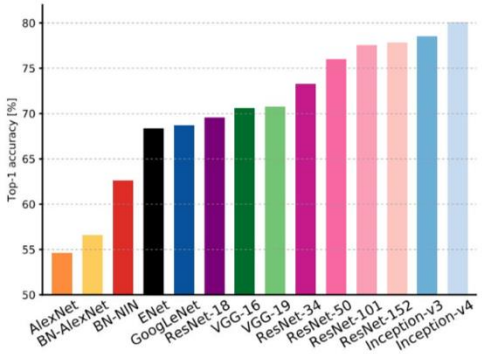
Version 04 April 2024

View Top 1 Accuracy by Date for All models



Top 1 Accuracy: 91.0%
(reported on paperswithcode)

CoCa: Contrastive Captioners are Image-Text Foundation Models (2022)



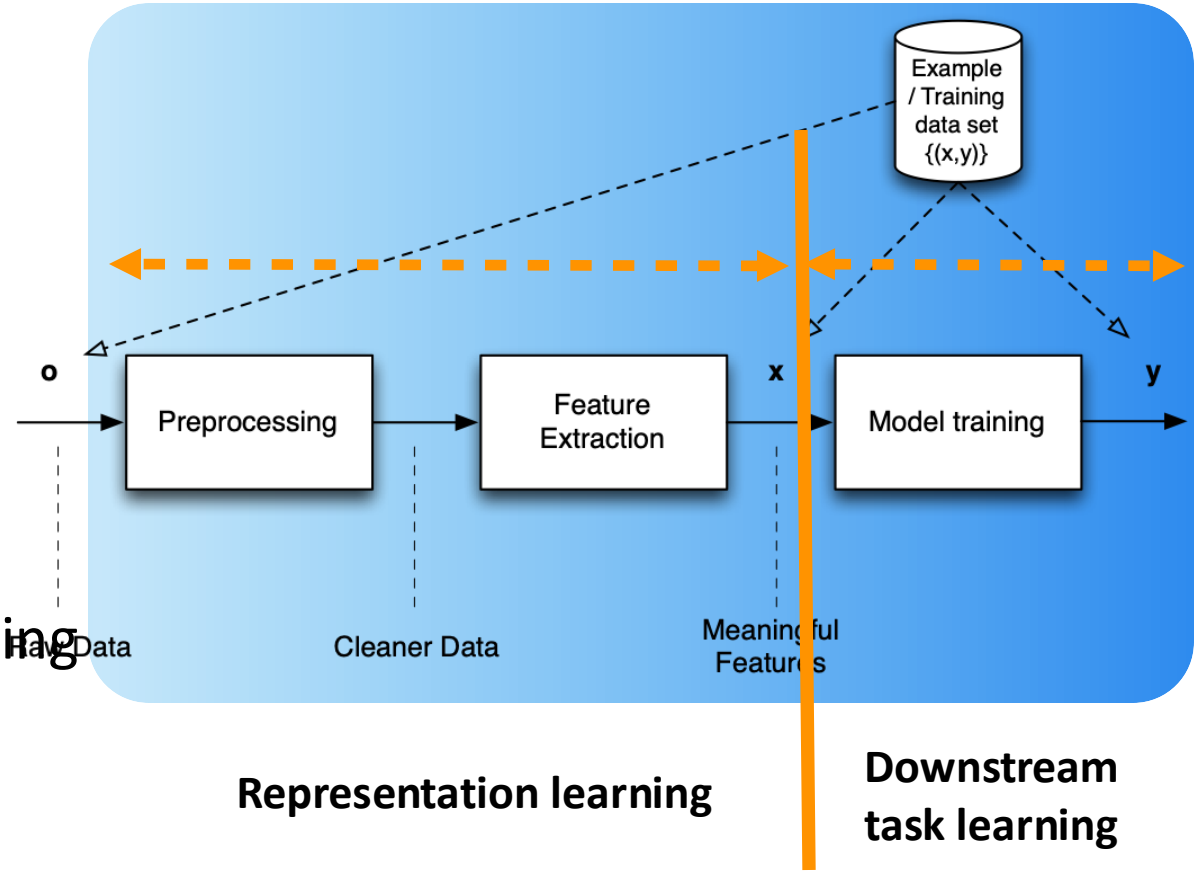
Transfer Learning

Idea

The filter parameters are learnt through the training process. The model is learning not only to classify but also to extract features. Use the feature extraction part for learning other use case cases.

Deep Learning:
Representation + Downstream Task Learning

Transfer Learning:
Reuse the representation learning part



Representation Learning

- Deep neural networks can be viewed as performing a two-stage process of first learning useful representations of the input, followed by a simple mapping to the target form.
- Representation learning can be viewed as an independent task, where the goal is to transform high-dimensional complex input into a low-dimensional form that captures information relevant to the given task, another task or several tasks.
  - Example: CNN that classifies high-dimensional natural images into several categories implicitly learns representations that transform non-linearly-separable input into a low-dimensional representation where the different classes can be linearly separated.
- Representation learning is a field of research and development aiming at creating models able to extract relevant (often generic) features of input data, capturing the underlying patterns without relying on manually designed features.
- This has received a lot of attention in recent years through self-supervised learning and so-called “Foundation Models”.

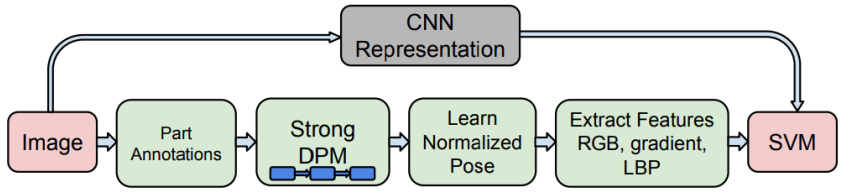
Transfer Learning

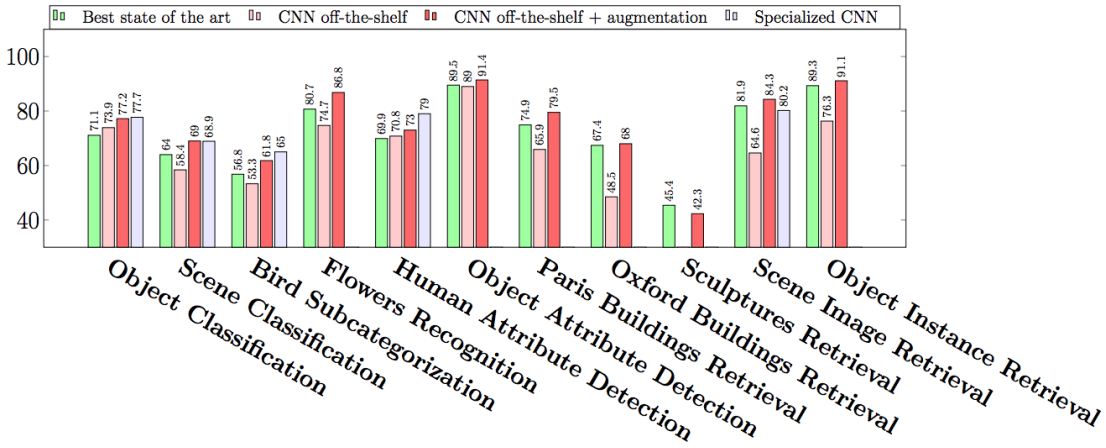
- Transfer learning is about
  - using knowledge learned from tasks trained on other dataset and
  - applying it to settings where only little labelled data is available.
- Re-use the feature extraction part (the “representation”) of pre-trained network on which years of engineering have been spent
- Transfer learning: kickstart the generalisation by starting from patterns learned for a different task

Transfer Learning - Beating State-of-the-Art

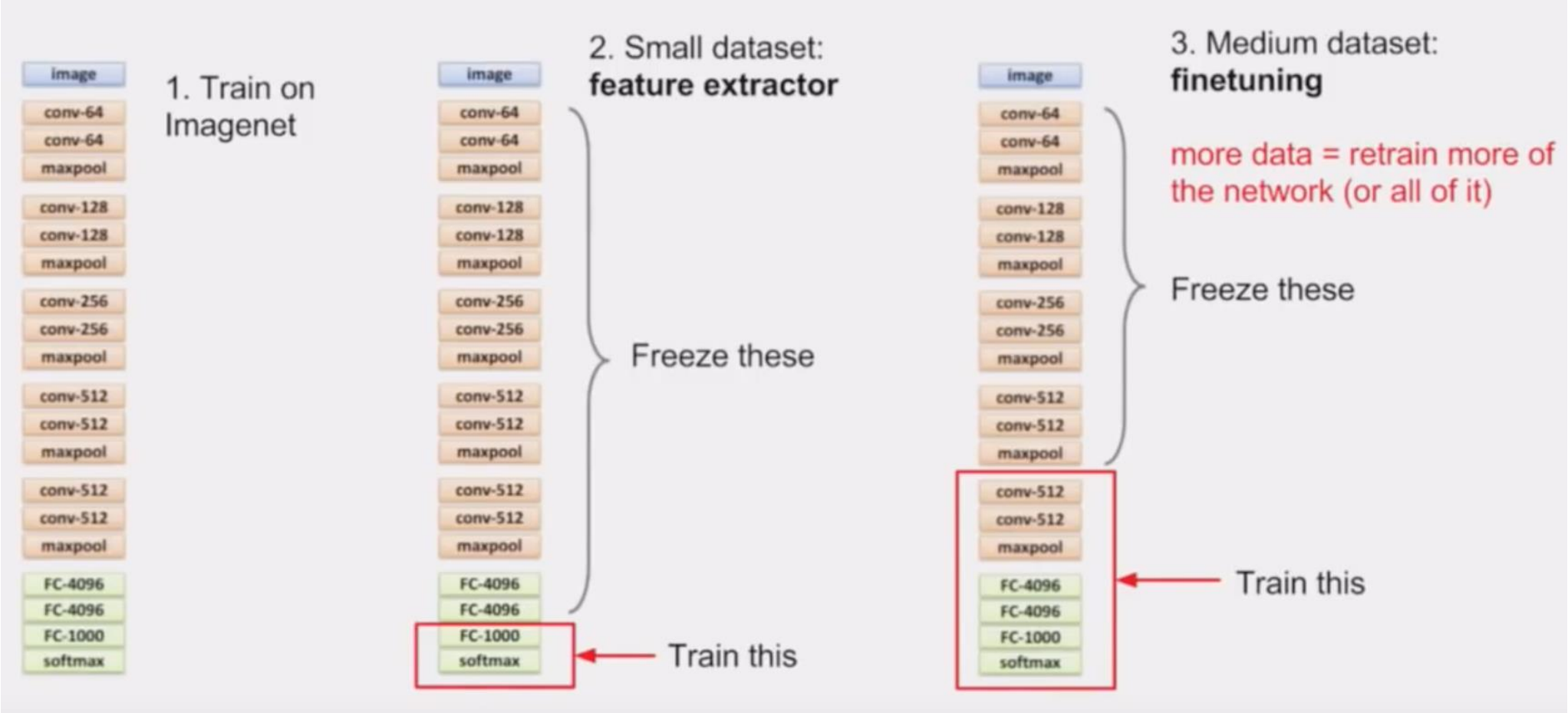
Paper: A. Razavian et al, "*CNN Features off-the-shelf: an Astounding Baseline for Recognition*", arXiv, 2014 https://arxiv.org/abs/1403.6382

"Recent results indicate that the generic descriptors extracted from the convolutional neural networks are very powerful. This paper adds to the mounting evidence that this is indeed the case. We report on a series of experiments conducted for different recognition tasks using the publicly available code and model of the OverFeat network which was trained to perform object classification on ILSVRC13. We use features extracted from the OverFeat network as a generic image representation to tackle the diverse range of recognition tasks of object image classification, scene recognition, fine grained recognition, attribute detection and image retrieval applied to a diverse set of datasets. We selected these tasks and datasets as they gradually move further away from the original task and data the OverFeat network was trained to solve. Astonishingly, we report consistent superior results compared to the highly tuned state-of-the-art systems in all the visual classification tasks on various datasets."





Transfer Learning – some practice



Transfer Learning

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4

in PyTorch many pre-trained models for image recognition available out of the box:
see
https://docs.pytorch.org/vision/main/models.html

- AlexNet
- ConvNeXt
- DenseNet
- EfficientNet
- EfficientNetV2
- GoogLeNet
- Inception V3
- MaxVit
- MNASNet
- MobileNet V2
- MobileNet V3
- RegNet
- ResNet
- ResNeXt
- ShuffleNet V2
- SqueezeNet
- SwinTransformer
- VGG
- VisionTransformer
- Wide ResNet

Some further references / pointers

- torch.hub:
  - pre-trained model repository designed to facilitate research reproducibility
  - instancing a pre-trained model will download its weights to a cache directory.
  - see e.g. torch.hub.load_state_dict_from_url() for details.
- hugging face:
  - see transformers package
  - extremely large repo of pretrained models (model cards)
- timm:
  - through hugging face
  - torch image models